Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №2
дисциплины
«Основы нейронных сетей»
Вариант № 14

Выполнила:
Текеева Мадина Азрет-Алиевна
3 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

_____
(подпись)

Проверил:
Доцент департамента цифровых, робототехнических систем и электроники института перспективной инженерии Воронкин Р.А

_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты_____

Ставрополь, 2026 г.

Тема работы: «Разработка и исследование нейронной сети для классификации изображений с применением методов регуляризации Dropout и Batch Normalization».

Цель работы: разработать и обучить нейронную сеть для классификации изображений предметов одежды, вин и пассажиров автобуса, исследовать влияние методов регуляризации Dropout и Batch Normalization на процесс обучения и качество модели.

Порядок выполнения работы

Задание 1.1 Используя шаблон ноутбука для распознавания видов одежды и аксессуаров из набора «fashion_mnist», выполнить следующие действия:

1) создать 9 моделей нейронной сети с различными архитектурами и сравнить в них значения точности на проверочной выборке (на последней эпохе) и на тестовой выборке. Использовать следующее деление: обучающая выборка – 50000 примеров, проверочная выборка – 10000 примеров, тестовая выборка – 10000 примеров;

2) создать сравнительную таблицу в конце ноутбука, написать выводы по результатам проведённых тестов.

1.1.1 Импорт необходимых библиотек для задания и загрузка датасета в переменные.

Листинг 1.1 – Импорт библиотек:

```
# Последовательная модель НС
from tensorflow.keras.models import Sequential
# Основные слои
from tensorflow.keras.layers import Dense, Activation, Dropout,
BatchNormalization
# Колбэк ранней остановки
from tensorflow.keras.callbacks import EarlyStopping
# Утилиты для to_categorical()
from tensorflow.keras import utils
# Алгоритмы оптимизации для обучения модели
```

```
from tensorflow.keras.optimizers import Adam, Adadelta
# Библиотека для работы с массивами
import numpy as np
# Библиотека для работы с таблицами
import pandas as pd
# Отрисовка графиков
import matplotlib.pyplot as plt
# Связь с google-диском
from google.colab import files
# Предварительная обработка данных
from sklearn import preprocessing
# Разделение данных на выборки
from sklearn.model_selection import train_test_split
# Для загрузки датасета
from keras.datasets import fashion_mnist
# Отрисовывать изображения в ноутбуке, а не в консоль или файл
%matplotlib inline
```

Листинг 1.2 – Загрузка датасета:

```
# Загрузка датасета
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
print('Размер x_train:', x_train.shape)
print('Размер y_train:', y_train.shape)
print('Размер x_test:', x_test.shape)
print('Размер y_test:', y_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 2us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Размер x_train: (60000, 28, 28)
Размер y_train: (60000,)
Размер x_test: (10000, 28, 28)
Размер y_test: (10000,)
```

Рисунок 1.1 – Размерности выборок после загрузки датасета fashion_mnist

Листинг 1.3 – Разделение обучающей выборки на обучающую (50000) и проверочную (10000):

```
x_train, x_val, y_train, y_val = train_test_split(
    x_train, y_train,
    test_size=10000,     # ровно 10000 примеров в валидацию
```

```
    shuffle=True,        # перемешиваем
    random_state=42      # воспроизводимость
)
print('Обучающая выборка:   ', x_train.shape, y_train.shape)
print('Валидационная:       ', x_val.shape,   y_val.shape)
print('Тестовая:            ', x_test.shape,  y_test.shape)
```

```
Обучающая выборка:   (50000, 28, 28) (50000,)
Валидационная:       (10000, 28, 28) (10000,)
Тестовая:            (10000, 28, 28) (10000,)
```

Рисунок 1.2 – Результат разделения на обучающую, проверочную и тестовую выборки

Листинг 1.4 – Нормализация и one-hot encoding меток:

```
# Преобразование к формату (N, 784) и нормализация в диапазон [0, 1]
x_train = x_train.reshape(x_train.shape[0], -1).astype('float32') / 255.
x_val   = x_val.reshape(x_val.shape[0], -1).astype('float32') / 255.
x_test  = x_test.reshape(x_test.shape[0], -1).astype('float32') / 255.
# One-hot encoding меток (10 классов)
y_train_cat = utils.to_categorical(y_train, 10)
y_val_cat   = utils.to_categorical(y_val,   10)
y_test_cat  = utils.to_categorical(y_test,  10)
```

Листинг 1.5 – Отрисовка 10 разных классов изображений из датасета:

```
# Выбор 1 изображения каждого класса
imgs = np.array([x_train[y_train==i][0] for i in range(10)])
imgs = np.concatenate(imgs, axis=1)
plt.figure(figsize=(30, 6))
plt.imshow(imgs, cmap='Greys_r')
plt.grid(False)
plt.axis('off')
plt.show()
```

```
# Выбор 1 изображения каждого класса
imgs = np.array([x_train[y_train==i][0] for i in range(10)])

# Соединение изображения в одну линию
imgs = np.concatenate(imgs, axis=1)

# Создание поля для изображения
plt.figure(figsize=(30, 6))

# Отрисовка итогового изображения
plt.imshow(imgs, cmap='Greys_r')

# Без сетки
plt.grid(False)

# Без осей
plt.axis('off')

# Вывод результата
plt.show()
```

Рисунок 1.3 – Примеры изображений каждого класса из датасета fashion_mnist

1.1.2 Построение вспомогательной функции для обучения и оценки моделей.

Для того чтобы не дублировать код при обучении 9 моделей, была написана вспомогательная функция train_and_evaluate, которая выполняет обучение модели с применением колбэка EarlyStopping, забирает значение val_accuracy на последней эпохе обучения согласно заданию, вычисляет точность модели на тестовой выборке методом .evaluate() и возвращает результаты для формирования итоговой сравнительной таблицы.

Листинг 1.6 – Вспомогательная функция train_and_evaluate:

```
def train_and_evaluate(model, name, epochs=30, batch_size=256, verbose=0):
    """Обучает модель с EarlyStopping и возвращает точность на val и test."""

    early_stop = EarlyStopping(
        monitor='val_loss',
        mode='min',
        patience=3,
        restore_best_weights=True
    )

    history = model.fit(
        x_train, y_train_cat,
        batch_size=batch_size,
```

```
        epochs=epochs,
        validation_data=(x_val, y_val_cat),
        callbacks=[early_stop],
        verbose=verbose
    )
    val_acc_last = history.history['val_accuracy'][-1]
    test_loss, test_acc = model.evaluate(x_test, y_test_cat, verbose=0)
    epochs_done = len(history.history['loss'])
    print(f'[{name}]  эпох: {epochs_done}   '
          f'val_accuracy: {val_acc_last:.4f}   '
          f'test_accuracy: {test_acc:.4f}')

    return {
        'name': name,
        'val_accuracy': val_acc_last,
        'test_accuracy': test_acc,
        'epochs_done': epochs_done,
        'history': history
    }

results = []
```

1.1.3 Построение однослойной модели нейронной сети с 128 нейронами.

Минимально простая архитектура: один скрытый слой с 128 нейронами и функцией активации ReLU. Выходной слой состоит из 10 нейронов с функцией активации Softmax. Согласно теории, при решении задачи рекомендуется начинать именно с минимально простой архитектуры.

Листинг 1.7 – Создание модели 1 (архитектура 128–10)

```
model_1 = Sequential()
model_1.add(Dense(128, input_dim=784, activation='relu'))
model_1.add(Dense(10, activation='softmax'))
model_1.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])
```

```
results.append(train_and_evaluate(model_1, 'M1: однослойная 128'))
```

```
model_1 = Sequential()
model_1.add(Dense(128, input_dim=784, activation='relu'))
model_1.add(Dense(10, activation='softmax'))
model_1.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])

results.append(train_and_evaluate(model_1, 'M1: однослойная 128'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argumen
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
[M1: однослойная 128]  эпох: 14   val_accuracy (посл. эпоха): 0.8760   test_accuracy: 0.8765
```

Рисунок 1.4 – Результат обучения модели 1

1.1.4 Построение однослойной модели нейронной сети с 256 нейронами.

Та же архитектура, но с увеличенным числом нейронов в скрытом слое до 256 – для проверки того, даёт ли расширение слоя прирост точности.

Листинг 1.8 – Создание модели 2 (архитектура 256–10)

```
model_2 = Sequential()
model_2.add(Dense(256, input_dim=784, activation='relu'))
model_2.add(Dense(10, activation='softmax'))
model_2.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])

results.append(train_and_evaluate(model_2, 'M2: однослойная 256'))
```

```
model_2 = Sequential()
model_2.add(Dense(256, input_dim=784, activation='relu'))
model_2.add(Dense(10, activation='softmax'))
model_2.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])

results.append(train_and_evaluate(model_2, 'M2: однослойная 256'))
```

```
[M2: однослойная 256]  эпох: 20   val_accuracy (посл. эпоха): 0.8903   test_accuracy: 0.8850
```

Рисунок 1.5 – Результат обучения модели 2

1.1.5 Построение двуслойной модели нейронной сети.

Добавлен второй скрытый слой. Первый скрытый слой содержит 128 нейронов, второй – 64 нейрона, оба используют функцию активации ReLU. Выходной слой состоит из 10 нейронов с функцией активации Softmax.

Листинг 1.9 – Создание модели 3 (архитектура 128–64–10)

```
model_3 = Sequential()
model_3.add(Dense(128, input_dim=784, activation='relu'))
model_3.add(Dense(64, activation='relu'))
model_3.add(Dense(10, activation='softmax'))
model_3.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])


results.append(train_and_evaluate(model_3, 'M3: двуслойная 128 → 64'))
```

```
model_3 = Sequential()
model_3.add(Dense(128, input_dim=784, activation='relu'))
model_3.add(Dense(64, activation='relu'))
model_3.add(Dense(10, activation='softmax'))
model_3.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])

results.append(train_and_evaluate(model_3, 'M3: двуслойная 128 → 64'))

[M3: двуслойная 128 → 64]  эпох: 17   val_accuracy (посл. эпоха): 0.8860   test_accuracy: 0.8827
```

Рисунок 1.6 – Результат обучения модели 3

1.1.6 Построение двуслойной модели нейронной сети.

Более широкая двуслойная сеть: первый скрытый слой содержит 256 нейронов, второй – 128 нейронов, оба с функцией активации ReLU. Выходной слой состоит из 10 нейронов с функцией активации Softmax.

Листинг 1.10 – Создание модели 4 (архитектура 256–128–10)

```
model_4 = Sequential()
model_4.add(Dense(256, input_dim=784, activation='relu'))
model_4.add(Dense(128, activation='relu'))
model_4.add(Dense(10, activation='softmax'))
model_4.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
```

```
                metrics=['accuracy'])

results.append(train_and_evaluate(model_4, 'M4: двуслойная 256 → 128'))
```

```
model_4 = Sequential()
model_4.add(Dense(256, input_dim=784, activation='relu'))
model_4.add(Dense(128, activation='relu'))
model_4.add(Dense(10, activation='softmax'))
model_4.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])

results.append(train_and_evaluate(model_4, 'M4: двуслойная 256 → 128'))

[M4: двуслойная 256 → 128]  эпох: 17   val_accuracy (посл. эпоха): 0.8871   test_accuracy: 0.8867
```

Рисунок 1.7 – Результат обучения модели 4

1.1.7 Построение трёхслойной модели нейронной сети.

Глубокая сеть с тремя скрытыми слоями. Первый слой содержит 512 нейронов, второй – 256 нейронов, третий – 128 нейронов. Во всех скрытых слоях применяется функция активации ReLU. Данная модель позволяет проверить, помогает ли увеличение глубины сети улучшить качество классификации.

Листинг 1.11 – Создание модели 5 (архитектура 512–256–128–10)

```
model_5 = Sequential()
model_5.add(Dense(512, input_dim=784, activation='relu'))
model_5.add(Dense(256, activation='relu'))
model_5.add(Dense(128, activation='relu'))
model_5.add(Dense(10, activation='softmax'))
model_5.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])

results.append(train_and_evaluate(model_5, 'M5: трёхслойная 512 → 256 →
128'))
```

```
model_5 = Sequential()
model_5.add(Dense(512, input_dim=784, activation='relu'))
model_5.add(Dense(256, activation='relu'))
model_5.add(Dense(128, activation='relu'))
model_5.add(Dense(10, activation='softmax'))
model_5.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])

results.append(train_and_evaluate(model_5, 'M5: трёхслойная 512 → 256 → 128'))
```

```
[M5: трёхслойная 512 → 256 → 128]  эпох: 10   val_accuracy (посл. эпоха): 0.8854   test_accuracy: 0.8809
```

Рисунок 1.8 – Результат обучения модели 5

1.1.8 Построение двуслойной модели нейронной сети с регуляризацией Dropout.

Двуслойная сеть 256 → 128 с добавлением слоёв регуляризации Dropout с коэффициентом 0.3 после каждого скрытого слоя. Согласно теории, Dropout случайно отключает часть нейронов в каждом батче, что является одним из основных методов борьбы с переобучением.

Листинг 1.12 – Создание модели 6 (архитектура с Dropout)

```
model_6 = Sequential()
model_6.add(Dense(256, input_dim=784, activation='relu'))
model_6.add(Dropout(0.3))
model_6.add(Dense(128, activation='relu'))
model_6.add(Dropout(0.3))
model_6.add(Dense(10, activation='softmax'))
model_6.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])

results.append(train_and_evaluate(model_6, 'M6: 256 → Dropout → 128 →
Dropout'))
```

```
model_6 = Sequential()
model_6.add(Dense(256, input_dim=784, activation='relu'))
model_6.add(Dropout(0.3))
model_6.add(Dense(128, activation='relu'))
model_6.add(Dropout(0.3))
model_6.add(Dense(10, activation='softmax'))
model_6.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])

results.append(train_and_evaluate(model_6, 'M6: 256 → Dropout → 128 → Dropout'))
```

```
[M6: 256 → Dropout → 128 → Dropout]  эпох: 19   val_accuracy (посл. эпоха): 0.8928   test_accuracy: 0.8869
```

Рисунок 1.9 – Результат обучения модели 6

1.1.9 Построение двуслойной модели нейронной сети с BatchNormalization.

Двуслойная сеть 256 → 128 с добавлением слоёв пакетной нормализации BatchNormalization после каждого скрытого слоя. Пакетная нормализация стандартизирует распределение выходов нейронов, позволяет увеличить шаг обучения и уменьшает чувствительность к начальным весам.

Листинг 1.13 – Создание модели 7 (архитектура с BatchNormalization)

```
model_7 = Sequential()
model_7.add(Dense(256, input_dim=784, activation='relu'))
model_7.add(BatchNormalization())
model_7.add(Dense(128, activation='relu'))
model_7.add(BatchNormalization())
model_7.add(Dense(10, activation='softmax'))
model_7.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])
results.append(train_and_evaluate(model_7, 'M7: 256 → BN → 128 → BN'))
```

```
model_7 = Sequential()
model_7.add(Dense(256, input_dim=784, activation='relu'))
model_7.add(BatchNormalization())
model_7.add(Dense(128, activation='relu'))
model_7.add(BatchNormalization())
model_7.add(Dense(10, activation='softmax'))
model_7.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])

results.append(train_and_evaluate(model_7, 'M7: 256 → BN → 128 → BN'))
```

```
[M7: 256 → BN → 128 → BN]  эпох:  8   val_accuracy (посл. эпоха): 0.8479   test_accuracy: 0.8572
```

Рисунок 1.10 – Результат обучения модели 7

1.1.10 Построение глубокой модели нейронной сети с Dropout и BatchNormalization.

Комбинация двух инструментов регуляризации – Dropout и BatchNormalization – применяемых в трёхслойной сети 512 → 256 → 128. Такая архитектура позволяет одновременно стандартизировать распределения и случайно отключать нейроны для борьбы с переобучением.

Листинг 1.14 – Создание модели 8 (глубокая + BN + Dropout)

```
model_8 = Sequential()
model_8.add(Dense(512, input_dim=784, activation='relu'))
model_8.add(BatchNormalization())
model_8.add(Dropout(0.3))
model_8.add(Dense(256, activation='relu'))
model_8.add(BatchNormalization())
model_8.add(Dropout(0.3))
model_8.add(Dense(128, activation='relu'))
model_8.add(Dropout(0.2))
model_8.add(Dense(10, activation='softmax'))
model_8.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])
results.append(train_and_evaluate(model_8, 'M8: глубокая + BN + Dropout'))
```

```
model_8 = Sequential()
model_8.add(Dense(512, input_dim=784, activation='relu'))
model_8.add(BatchNormalization())
model_8.add(Dropout(0.3))
model_8.add(Dense(256, activation='relu'))
model_8.add(BatchNormalization())
model_8.add(Dropout(0.3))
model_8.add(Dense(128, activation='relu'))
model_8.add(Dropout(0.2))
model_8.add(Dense(10, activation='softmax'))
model_8.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])

results.append(train_and_evaluate(model_8, 'M8: глубокая + BN + Dropout'))
```

```
[M8: глубокая + BN + Dropout]  эпох: 12   val_accuracy (посл. эпоха): 0.8772   test_accuracy: 0.8694
```

Рисунок 1.11 – Результат обучения модели 8

1.1.11 Построение двуслойной модели нейронной сети с оптимизатором Adadelta.

Для проверки влияния гиперпараметра «алгоритм оптимизации» была построена модель с архитектурой, идентичной модели 4 (256 → 128), но с применением оптимизатора Adadelta вместо Adam. Это позволяет оценить, как выбор оптимизатора влияет на сходимость и итоговую точность.

Листинг 1.15 – Создание модели 9 (архитектура 256–128 с Adadelta)

```
model_9 = Sequential()
model_9.add(Dense(256, input_dim=784, activation='relu'))
model_9.add(Dense(128, activation='relu'))
model_9.add(Dense(10, activation='softmax'))
model_9.compile(loss='categorical_crossentropy',
                optimizer=Adadelta(learning_rate=1.0),
                metrics=['accuracy'])
results.append(train_and_evaluate(model_9, 'M9: 256 → 128 (Adadelta)'))
```

```
model_9 = Sequential()
model_9.add(Dense(256, input_dim=784, activation='relu'))
model_9.add(Dense(128, activation='relu'))
model_9.add(Dense(10, activation='softmax'))
model_9.compile(loss='categorical_crossentropy',
                optimizer=Adadelta(learning_rate=1.0),
                metrics=['accuracy'])

results.append(train_and_evaluate(model_9, 'M9: 256 → 128 (Adadelta)'))
```

```
[M9: 256 → 128 (Adadelta)]  эпох:  8   val_accuracy (посл. эпоха): 0.8467   test_accuracy: 0.8663
```

Рисунок 1.12 – Результат обучения модели 9

1.1.12 Формирование сравнительной таблицы и визуализация результатов.

Для сравнения всех обученных моделей была составлена сводная таблица и построены столбчатая диаграмма, а также сводный график динамики val_accuracy по эпохам для всех моделей.

Листинг 1.16 – Формирование сравнительной таблицы

```
df_results = pd.DataFrame([
    {
        '№': i + 1,
        'Архитектура': r['name'],
```

```
        'Эпох пройдено': r['epochs_done'],
        'val_accuracy (посл. эпоха)': round(r['val_accuracy'], 4),
        'test_accuracy': round(r['test_accuracy'], 4),
    }
    for i, r in enumerate(results)
])


df_results_sorted = df_results.sort_values('test_accuracy',
ascending=False).reset_index(drop=True)
print(df_results.to_string(index=False))
print(df_results_sorted.to_string(index=False))
```

```
=== Результаты в порядке добавления ===
 №                      Архитектура  Эпох пройдено  val_accuracy (посл. эпоха)  test_accuracy
 1              M1: однослойная 128             14                      0.8760         0.8765
 2              M2: однослойная 256             20                      0.8903         0.8850
 3          M3: двуслойная 128 → 64             17                      0.8860         0.8827
 4         M4: двуслойная 256 → 128             17                      0.8871         0.8867
 5   M5: трёхслойная 512 → 256 → 128            10                      0.8854         0.8809
 6 M6: 256 → Dropout → 128 → Dropout            19                      0.8928         0.8869
 7          M7: 256 → BN → 128 → BN              8                      0.8479         0.8572
 8      M8: глубокая + BN + Dropout             12                      0.8772         0.8694
 9         M9: 256 → 128 (Adadelta)              8                      0.8467         0.8663

=== Результаты, отсортированные по test_accuracy ===
 №                      Архитектура  Эпох пройдено  val_accuracy (посл. эпоха)  test_accuracy
 6 M6: 256 → Dropout → 128 → Dropout            19                      0.8928         0.8869
 4         M4: двуслойная 256 → 128             17                      0.8871         0.8867
 2              M2: однослойная 256             20                      0.8903         0.8850
 3          M3: двуслойная 128 → 64             17                      0.8860         0.8827
 5   M5: трёхслойная 512 → 256 → 128            10                      0.8854         0.8809
 1              M1: однослойная 128             14                      0.8760         0.8765
 8      M8: глубокая + BN + Dropout             12                      0.8772         0.8694
 9         M9: 256 → 128 (Adadelta)              8                      0.8467         0.8663
 7          M7: 256 → BN → 128 → BN              8                      0.8479         0.8572
```

Рисунок 1.13 – Сравнительная таблица результатов

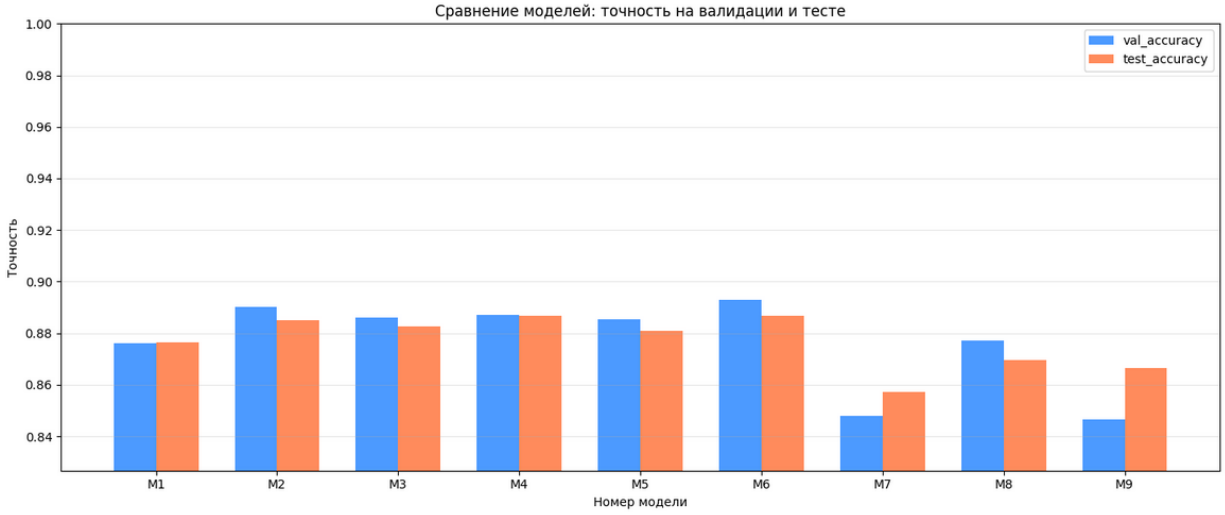


Рисунок 1.14 – Столбчатая диаграмма сравнения моделей по точности

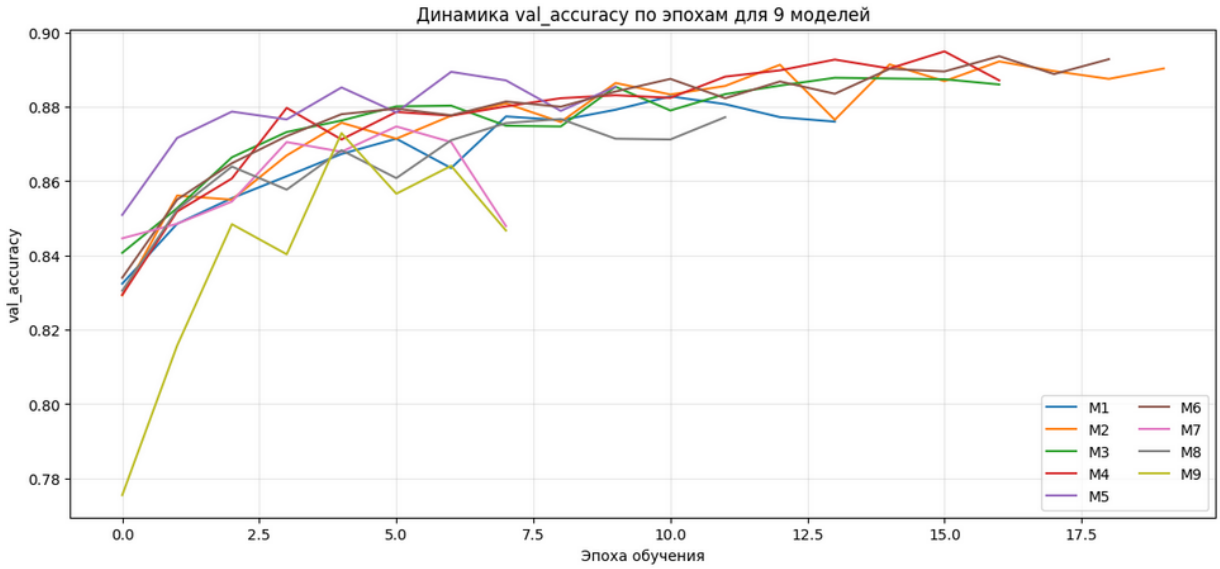


Рисунок 1.15 – Динамика val_accuracy по эпохам для 9 моделей

Таблица 1.1 – Сравнение 9 моделей нейронных сетей

№	Архитектура	Эпох	val_accuracy	test_accuracy
1	M1: однослойная 128	14	0.8760	0.8765
2	M2: однослойная 256	20	0.8903	0.8850
3	M3: двуслойная 128 → 64	17	0.8860	0.8827
4	M4: двуслойная 256 → 128	17	0.8871	0.8867
5	M5: трёхслойная 512 → 256 → 128	10	0.8854	0.8809
6	M6: 256 → Dropout → 128 → Dropout	19	0.8928	0.8869
7	M7: 256 → BN → 128 → BN	8	0.8479	0.8572
8	M8: глубокая + BN + Dropout	12	0.8772	0.8694
9	M9: 256 → 128 (Adadelta)	8	0.8467	0.8663

1.1.13 Выводы по заданию 1

По результатам обучения 9 моделей можно сформулировать следующие наблюдения:

Простые архитектуры не уступают сложным. Модели 1 и 2 (однослойные) показывают результат, сопоставимый с более глубокими сетями, что согласуется с рекомендацией из теории – начинать с минимально простой архитектуры и усложнять её постепенно.

Увеличение числа слоёв само по себе не гарантирует улучшения. Модель 5 с тремя скрытыми слоями (512 → 256 → 128) не даёт существенного прироста относительно двуслойных моделей, но обучается дольше и склонна к переобучению.

Dropout помогает бороться с переобучением. Модель 6 показывает лучший результат на валидации – val_accuracy = 0.8928, что является наивысшим значением среди всех моделей. Это подтверждает роль Dropout как эффективного регуляризатора.

BatchNormalization на данной задаче требует дополнительной настройки. Модель 7 с BatchNormalization показала более низкий результат (test_accuracy = 0.8572), что может быть связано с ранней остановкой обучения на 8 эпохе – сеть не успела стабилизироваться.

Оптимизатор существенно влияет на скорость сходимости. Модель 9 с Adadelta при той же архитектуре, что и модель 4, показала заметно меньшую точность (0.8663 против 0.8867) при том же числе эпох, что говорит о более медленной сходимости данного оптимизатора. Adam в большинстве случаев показывает себя лучше на стандартных задачах.

Применение колбэка EarlyStopping делает сравнение моделей честным – каждая модель обучается до момента, пока val_loss улучшается, после чего происходит восстановление лучших весов. Это исключает влияние случайных флуктуаций на последних эпохах.

Значения val_accuracy и test_accuracy у всех моделей близки, что говорит о корректности разделения выборок и адекватном подборе гиперпараметров. Лучший результат на тестовой выборке показала модель 6 (256 → Dropout → 128 → Dropout) с точностью 0.8869.

Задание 1.2 Используя шаблон ноутбука для распознавания видов одежды и аксессуаров из набора *fashion_mnist*, выполнить следующие действия используя pytorch:

– создать 9 моделей нейронной сети с различными архитектурами и сравнить в них значения точности на проверочной выборке (на последней

эпохе) и на тестовой выборке. Использовать следующее деление: обучающая выборка – 50000 примеров, проверочная выборка – 10000 примеров, тестовая выборка – 10000 примеров;

– создать сравнительную таблицу в конце ноутбука, написать выводы по результатам проведённых тестов.

1.2.1 Импорт необходимых библиотек для задания и загрузка датасета в переменные.

Для реализации задания используется фреймворк PyTorch. Импортируются модули torch, torch.nn и torch.optim для построения и обучения моделей, DataLoader и TensorDataset для батчевой подачи данных, модуль torchvision.datasets для загрузки датасета Fashion-MNIST, библиотеки numpy и pandas для работы с массивами и таблицами, matplotlib.pyplot для визуализации и sklearn.model_selection.train_test_split для разделения выборок. Дополнительно фиксируется seed для воспроизводимости и выбирается устройство (GPU при наличии, иначе CPU).

Листинг 1.1 – Импорт библиотек и выбор устройства:

```
# PyTorch
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

# Загрузка датасета
from torchvision import datasets, transforms

# Библиотека для работы с массивами
import numpy as np

# Библиотека для работы с таблицами
import pandas as pd

# Отрисовка графиков
import matplotlib.pyplot as plt
```

```
# Разделение данных на выборки
from sklearn.model_selection import train_test_split

# Отрисовывать изображения в ноутбуке
%matplotlib inline

# Фиксируем seed для воспроизводимости
SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)

# Выбор устройства (GPU если доступен)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Устройство:', device)
```

Импорт библиотек

```
# PyTorch
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

# Загрузка датасета
from torchvision import datasets, transforms

# Библиотека для работы с массивами
import numpy as np

# Библиотека для работы с таблицами
import pandas as pd

# Отрисовка графиков
import matplotlib.pyplot as plt

# Разделение данных на выборки
from sklearn.model_selection import train_test_split

# Отрисовывать изображения в ноутбуке
%matplotlib inline

# Фиксируем seed для воспроизводимости
SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)

# Выбор устройства (GPU если доступен)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Устройство:', device)

Устройство: cpu
```

Рисунок 1.1 – Импорт библиотек и выбор устройства

Датасет Fashion-MNIST содержит изображения одежды, обуви и аксессуаров размером 28×28 пикселей, разделённые на 10 классов: футболка, брюки, пуловер, платье, пальто, сандалии/босоножки, рубашка, кроссовки, сумочка, ботильоны. Обучающая выборка содержит 60000 примеров, тестовая – 10000 примеров. Загрузка выполняется через torchvision.datasets.FashionMNIST с автоматическим преобразованием к

тензорам и нормализацией в диапазон [0, 1]. Для удобства дальнейшей работы массивы numpy извлекаются из объектов датасета.

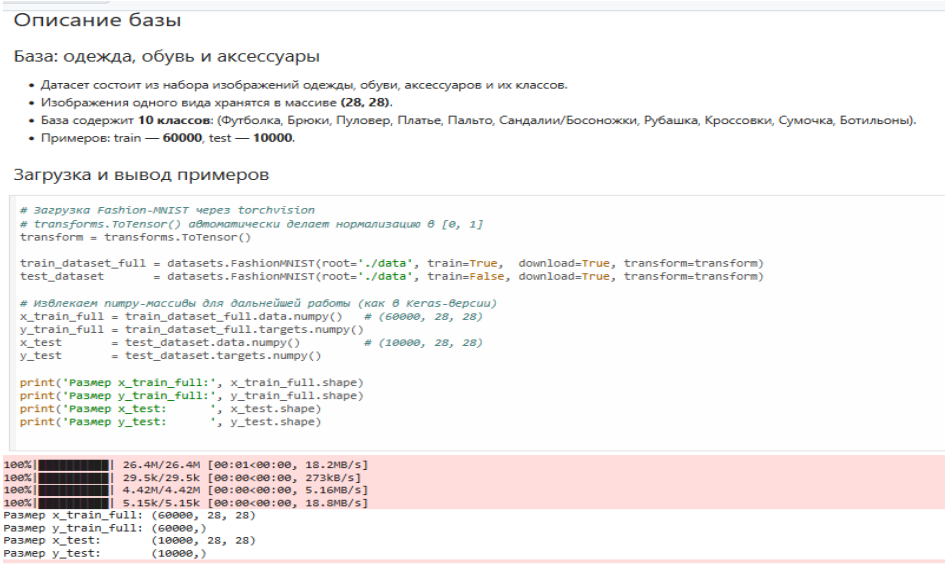
Листинг 1.2 – Загрузка датасета Fashion-MNIST:

```
# Загрузка Fashion-MNIST через torchvision
# transforms.ToTensor() автоматически делает нормализацию в [0, 1]
transform = transforms.ToTensor()

train_dataset_full = datasets.FashionMNIST(root='./data', train=True,
download=True, transform=transform)
test_dataset       = datasets.FashionMNIST(root='./data', train=False,
download=True, transform=transform)

# Извлекаем numpy-массивы для дальнейшей работы
x_train_full = train_dataset_full.data.numpy()       # (60000, 28, 28)
y_train_full = train_dataset_full.targets.numpy()
x_test       = test_dataset.data.numpy()             # (10000, 28, 28)
y_test       = test_dataset.targets.numpy()

print('Размер x_train_full:', x_train_full.shape)
print('Размер y_train_full:', y_train_full.shape)
print('Размер x_test:      ', x_test.shape)
print('Размер y_test:      ', y_test.shape)
```



Описание базы

База: одежда, обувь и аксессуары

- Датасет состоит из набора изображений одежды, обуви, аксессуаров и их классов.
- Изображения одного вида хранятся в массиве **(28, 28)**.
- База содержит **10 классов**: (Футболка, Брюки, Пуловер, Платье, Пальто, Сандалии/Босоножки, Рубашка, Кроссовки, Сумочка, Ботильоны).
- Примеров: train — **60000**, test — **10000**.

Загрузка и вывод примеров

```
# Загрузка Fashion-MNIST через torchvision
# transforms.ToTensor() автоматически делает нормализацию в [0, 1]
transform = transforms.ToTensor()

train_dataset_full = datasets.FashionMNIST(root='./data', train=True,  download=True, transform=transform)
test_dataset       = datasets.FashionMNIST(root='./data', train=False, download=True, transform=transform)

# Извлекаем numpy-массивы для дальнейшей работы (как в Keras-версии)
x_train_full = train_dataset_full.data.numpy()   # (60000, 28, 28)
y_train_full = train_dataset_full.targets.numpy()
x_test       = test_dataset.data.numpy()         # (10000, 28, 28)
y_test       = test_dataset.targets.numpy()

print('Размер x_train_full:', x_train_full.shape)
print('Размер y_train_full:', y_train_full.shape)
print('Размер x_test:      ', x_test.shape)
print('Размер y_test:      ', y_test.shape)
```

```
100%|██████████| 26.4M/26.4M [00:01<00:00, 18.2MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 273kB/s]
100%|██████████| 4.42M/4.42M [00:00<00:00, 5.16MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 18.8MB/s]
Размер x_train_full: (60000, 28, 28)
Размер y_train_full: (60000,)
Размер x_test:       (10000, 28, 28)
Размер y_test:       (10000,)
```

Рисунок 1.2 – Загрузка датасета и размерности массивов

Для визуальной проверки корректности загрузки данных выводится по одному изображению каждого из 10 классов. Изображения объединяются по горизонтали в одну линию и отрисовываются с помощью matplotlib в оттенках серого.

Листинг 1.3 – Отрисовка примеров 10 классов из датасета:

```
# Выбор 1 изображения каждого класса
imgs = np.array([x_train_full[y_train_full==i][0] for i in range(10)])

# Соединение изображений в одну линию
imgs = np.concatenate(imgs, axis=1)

# Создание поля для изображения
plt.figure(figsize=(30, 6))

# Отрисовка итогового изображения
plt.imshow(imgs, cmap='Greys_r')

plt.grid(False)
plt.axis('off')
plt.show()
```

Рисунок 1.3 – Примеры изображений каждого класса из датасета Fashion-MNIST

1.2.2 Подготовка выборок и формирование DataLoader'ов.

Согласно заданию, выборки делятся следующим образом: обучающая – 50000 примеров, проверочная – 10000 примеров, тестовая – 10000 примеров. Исходный набор Fashion-MNIST изначально содержит train = 60000 и test = 10000 примеров. Тестовая выборка берётся как есть, а обучающий набор делится на обучающую и валидационную части с помощью функции train_test_split из библиотеки scikit-learn.

Листинг 1.4 – Разделение выборок и преобразование в тензоры PyTorch:

```
# Разделение train-выборки на обучающую (50000) и проверочную (10000)
x_train, x_val, y_train, y_val = train_test_split(
    x_train_full, y_train_full,
    test_size=10000,     # ровно 10000 примеров в валидацию
    shuffle=True,        # перемешиваем
    random_state=42      # воспроизводимость
)

print('Обучающая выборка:  ', x_train.shape, y_train.shape)
print('Валидационная:      ', x_val.shape,   y_val.shape)
print('Тестовая:           ', x_test.shape,  y_test.shape)

# Преобразование к формату (N, 784) и нормализация в диапазон [0, 1]
x_train = x_train.reshape(x_train.shape[0], -1).astype('float32') / 255.
x_val   = x_val.reshape(x_val.shape[0],   -1).astype('float32') / 255.
x_test  = x_test.reshape(x_test.shape[0], -1).astype('float32') / 255.

# Перевод в тензоры PyTorch
x_train_t = torch.tensor(x_train, dtype=torch.float32)
x_val_t   = torch.tensor(x_val,   dtype=torch.float32)
x_test_t  = torch.tensor(x_test,  dtype=torch.float32)

# Метки как Long (для CrossEntropyLoss)
y_train_t = torch.tensor(y_train, dtype=torch.long)
y_val_t   = torch.tensor(y_val,   dtype=torch.long)
y_test_t  = torch.tensor(y_test,  dtype=torch.long)
```

```
Обучающая выборка:   (50000, 28, 28) (50000,)
Валидационная:       (10000, 28, 28) (10000,)
Тестовая:            (10000, 28, 28) (10000,)
```

Рисунок 1.4 – Разделение выборок

```
x_train_t: torch.Size([50000, 784])  y_train_t: torch.Size([50000])
x_val_t:   torch.Size([10000, 784])  y_val_t:   torch.Size([10000])
x_test_t:  torch.Size([10000, 784])  y_test_t:  torch.Size([10000])
```

Рисунок 1.5 – Преобразование в тензоры PyTorch

Для батчевого обучения создаются объекты DataLoader на основе TensorDataset. Размер батча выбран равным 256. Для обучающей выборки данные перемешиваются в каждой эпохе (shuffle=True), для проверочной и тестовой – нет.

Листинг 1.5 – Формирование DataLoader'ов для батчевого обучения:

```
# DataLoader'ы для батчевого обучения
BATCH_SIZE = 256

train_loader = DataLoader(TensorDataset(x_train_t, y_train_t),
                          batch_size=BATCH_SIZE, shuffle=True)

val_loader   = DataLoader(TensorDataset(x_val_t,   y_val_t),
                          batch_size=BATCH_SIZE, shuffle=False)

test_loader  = DataLoader(TensorDataset(x_test_t,  y_test_t),
                          batch_size=BATCH_SIZE, shuffle=False)
```

Рисунок 1.5 – Формирование DataLoader'ов

Важно отметить, что в PyTorch в качестве функции потерь используется nn.CrossEntropyLoss, которая внутри уже выполняет softmax. Поэтому последний слой всех моделей не содержит активации softmax – на выход подаются «сырые» логиты. Это стандартная практика PyTorch и аналог связки categorical_crossentropy + softmax из Keras.

1.2.3 Построение вспомогательных функций для обучения и оценки моделей.

Чтобы не дублировать код при обучении 9 моделей, написаны две вспомогательные функции: evaluate() – считает точность и значение функции потерь на любом DataLoader'е; train_and_evaluate() – обучает модель с аналогом EarlyStopping (останавливается, если val_loss не улучшается 3 эпохи подряд), восстанавливает лучшие веса и возвращает результаты.

Листинг 1.6 – Функция оценки модели evaluate():

```
def evaluate(model, loader, criterion):
```

```
    """Считает loss и accuracy на переданном DataLoader."""
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for x_batch, y_batch in loader:
            x_batch, y_batch = x_batch.to(device), y_batch.to(device)
            logits = model(x_batch)
            loss = criterion(logits, y_batch)
            total_loss += loss.item() * x_batch.size(0)
            preds = logits.argmax(dim=1)
            correct += (preds == y_batch).sum().item()
            total += y_batch.size(0)
    return total_loss / total, correct / total
```

Листинг 1.7 – Функция обучения train_and_evaluate() с EarlyStopping:

```
def train_and_evaluate(model, name, optimizer=None, epochs=30, patience=3,
verbose=False):
    """Обучает модель с EarlyStopping и возвращает точность на val (посл.
эпоха) и на test."""
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    if optimizer is None:
        optimizer = optim.Adam(model.parameters(), lr=0.001)

    history = {'loss': [], 'val_loss': [], 'accuracy': [], 'val_accuracy':
[]}

    best_val_loss = float('inf')
    best_state = None
    epochs_no_improve = 0
    epochs_done = 0

    for epoch in range(epochs):
        # --- тренировка ---
```

```
        model.train()
        train_loss = 0.0
        correct = 0
        total = 0
        for x_batch, y_batch in train_loader:
            x_batch, y_batch = x_batch.to(device), y_batch.to(device)
            optimizer.zero_grad()
            logits = model(x_batch)
            loss = criterion(logits, y_batch)
            loss.backward()
            optimizer.step()
            train_loss += loss.item() * x_batch.size(0)
            correct += (logits.argmax(dim=1) == y_batch).sum().item()
            total += y_batch.size(0)
        train_loss /= total
        train_acc = correct / total

        # --- валидация ---
        val_loss, val_acc = evaluate(model, val_loader, criterion)

        history['loss'].append(train_loss)
        history['accuracy'].append(train_acc)
        history['val_loss'].append(val_loss)
        history['val_accuracy'].append(val_acc)
        epochs_done += 1

        if verbose:
            print(f'  Эпоха {epoch+1:>2}: loss={train_loss:.4f}
acc={train_acc:.4f} '
                  f'val_loss={val_loss:.4f} val_acc={val_acc:.4f}')

        # --- аналог EarlyStopping с restore_best_weights ---
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            best_state = {k: v.clone() for k, v in
model.state_dict().items()}
            epochs_no_improve = 0
```

```
        else:
            epochs_no_improve += 1
            if epochs_no_improve >= patience:
                break

    # Восстанавливаем лучшие веса
    if best_state is not None:
        model.load_state_dict(best_state)

    # Точность на валидации на последней эпохе обучения (по заданию)
    val_acc_last = history['val_accuracy'][-1]

    # Точность на тесте
    _, test_acc = evaluate(model, test_loader, criterion)

    print(f'[{name}]  эпох: {epochs_done:>2}   '
          f'val_accuracy (посл. эпоха): {val_acc_last:.4f}   '
          f'test_accuracy: {test_acc:.4f}')

    return {
        'name': name,
        'val_accuracy': val_acc_last,
        'test_accuracy': test_acc,
        'epochs_done': epochs_done,
        'history': history
    }

# Список для результатов всех моделей
results = []
```

1.2.4 Построение однослойной модели нейронной сети с 128 нейронами.

Минимально простая архитектура: один скрытый слой с 128 нейронами и функцией активации ReLU. Выходной слой состоит из 10 нейронов без softmax (softmax уже встроен в CrossEntropyLoss). Согласно теории, при

решении задачи рекомендуется начинать именно с минимально простой архитектуры.

Листинг 1.8 – Создание модели 1 (архитектура 784 → 128 → 10):

```
model_1 = nn.Sequential(
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Linear(128, 10)   # без softmax – он внутри CrossEntropyLoss
)
results.append(train_and_evaluate(model_1, 'M1: однослойная 128'))
```

```
[M1: однослойная 128]  эпох: 23   val_accuracy (посл. эпоха): 0.8897   test_accuracy: 0.8827
```

Рисунок 1.6 – Обучение модели 1

1.2.5 Построение однослойной модели нейронной сети с 256 нейронами.

Та же архитектура, что и в модели 1, но с увеличенным числом нейронов в скрытом слое до 256 – для проверки того, даёт ли расширение слоя прирост точности.

Листинг 1.9 – Создание модели 2 (архитектура 784 → 256 → 10):

```
model_2 = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Linear(256, 10)
)
results.append(train_and_evaluate(model_2, 'M2: однослойная 256'))
```

```
[M2: однослойная 256]  эпох: 16   val_accuracy (посл. эпоха): 0.8865   test_accuracy: 0.8843
```

Рисунок 1.7 – Обучение модели 2

1.2.6 Построение двуслойной модели нейронной сети (128 → 64).

Добавлен второй скрытый слой. Первый скрытый слой содержит 128 нейронов, второй – 64 нейрона, оба используют функцию активации ReLU. Выходной слой состоит из 10 нейронов.

Листинг 1.10 – Создание модели 3 (архитектура 784 → 128 → 64 → 10):

```
model_3 = nn.Sequential(
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 10)
)
results.append(train_and_evaluate(model_3, 'M3: двуслойная 128 → 64'))
```

```
[M3: двуслойная 128 → 64]  эпох: 12   val_accuracy (посл. эпоха): 0.8663   test_accuracy: 0.8685
```

Рисунок 1.8 – Обучение модели 3

1.2.7 Построение двуслойной модели нейронной сети (256 → 128).

Более широкая двуслойная сеть: первый скрытый слой содержит 256 нейронов, второй – 128 нейронов, оба с функцией активации ReLU. Выходной слой состоит из 10 нейронов.

Листинг 1.11 – Создание модели 4 (архитектура 784 → 256 → 128 → 10):

```
model_4 = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Linear(128, 10)
)
results.append(train_and_evaluate(model_4, 'M4: двуслойная 256 → 128'))
```

```
[M4: двуслойная 256 → 128]  эпох: 10   val_accuracy (посл. эпоха): 0.8762   test_accuracy: 0.8788
```

Рисунок 1.9 – Обучение модели 4

1.2.8 Построение трёхслойной модели нейронной сети.

Глубокая сеть с тремя скрытыми слоями. Первый слой содержит 512 нейронов, второй – 256 нейронов, третий – 128 нейронов. Во всех скрытых слоях применяется функция активации ReLU. Данная модель позволяет проверить, помогает ли увеличение глубины сети улучшить качество классификации.

Листинг 1.12 – Создание модели 5 (архитектура 784 → 512 → 256 → 128 → 10):

```
model_5 = nn.Sequential(
    nn.Linear(784, 512),
    nn.ReLU(),
    nn.Linear(512, 256),
    nn.ReLU(),
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Linear(128, 10)
)
results.append(train_and_evaluate(model_5, 'M5: трёхслойная 512 → 256 → 128'))
```

```
[M5: трёхслойная 512 → 256 → 128]  эпох: 10   val_accuracy (посл. эпоха): 0.8832   test_accuracy: 0.8807
```

Рисунок 1.10 – Обучение модели 5

1.2.9 Построение двуслойной модели нейронной сети с регуляризацией Dropout.

Двуслойная сеть 256 → 128 с добавлением слоёв регуляризации Dropout с коэффициентом 0.3 после каждого скрытого слоя. Согласно теории, Dropout случайно отключает часть нейронов в каждом батче, что является одним из основных методов борьбы с переобучением.

Листинг 1.13 – Создание модели 6 (архитектура с Dropout):

```
model_6 = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(256, 128),
```

```
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(128, 10)
)
results.append(train_and_evaluate(model_6, 'M6: 256 → Dropout → 128 →
Dropout'))
```

```
[M6: 256 → Dropout → 128 → Dropout]  эпох: 28   val_accuracy (посл. эпоха): 0.8945   test_accuracy: 0.8930
```

Рисунок 1.11 – Обучение модели 6

1.2.10 Построение двуслойной модели нейронной сети с BatchNormalization.

Двуслойная сеть 256 → 128 с добавлением слоёв пакетной нормализации BatchNormalization после каждого скрытого слоя. Пакетная нормализация стандартизирует распределение выходов нейронов, позволяет увеличить шаг обучения и уменьшает чувствительность к начальным весам. В PyTorch для полносвязных слоёв используется nn.BatchNorm1d(num_features).

Листинг 1.14 – Создание модели 7 (архитектура с BatchNormalization):

```
model_7 = nn.Sequential(
    nn.Linear(784, 256),
    nn.BatchNorm1d(256),
    nn.ReLU(),
    nn.Linear(256, 128),
    nn.BatchNorm1d(128),
    nn.ReLU(),
    nn.Linear(128, 10)
)
results.append(train_and_evaluate(model_7, 'M7: 256 → BN → 128 → BN'))
```

```
[M7: 256 → BN → 128 → BN]  эпох:  6   val_accuracy (посл. эпоха): 0.8545   test_accuracy: 0.8644
```

Рисунок 1.11 – Обучение модели 7

1.2.11 Построение глубокой модели нейронной сети с Dropout и BatchNormalization.

Комбинация двух инструментов регуляризации – Dropout и BatchNormalization – применяемых в трёхслойной сети 512 → 256 → 128. Такая архитектура позволяет одновременно стандартизировать распределения и случайно отключать нейроны для борьбы с переобучением.

Листинг 1.15 – Создание модели 8 (глубокая + BN + Dropout):

```
model_8 = nn.Sequential(
    nn.Linear(784, 512),
    nn.BatchNorm1d(512),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(512, 256),
    nn.BatchNorm1d(256),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(128, 10)
)
results.append(train_and_evaluate(model_8, 'M8: глубокая + BN + Dropout'))
```

```
[M8: глубокая + BN + Dropout]  эпох: 14   val_accuracy (посл. эпоха): 0.8873   test_accuracy: 0.8822
```

Рисунок 1.12 – Результат обучения модели 8

1.2.12 Построение двуслойной модели нейронной сети с оптимизатором Adadelta.

Для проверки влияния гиперпараметра «алгоритм оптимизации» была построена модель с архитектурой, идентичной модели 4 (256 → 128), но с применением оптимизатора Adadelta вместо Adam. Это позволяет оценить, как выбор оптимизатора влияет на сходимость и итоговую точность.

Листинг 1.16 – Создание модели 9 (архитектура 256 → 128 с Adadelta):

```
model_9 = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
```

```
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Linear(128, 10)
)

optimizer_9 = optim.Adadelta(model_9.parameters(), lr=1.0)
results.append(train_and_evaluate(model_9, 'M9: 256 → 128 (Adadelta)',
                                  optimizer=optimizer_9))
```

```
[M9: 256 → 128 (Adadelta)]  эпох: 14   val_accuracy (посл. эпоха): 0.8793   test_accuracy: 0.8788
```

Рисунок 1.13 –Результат обучения модели 9

1.2.13 Формирование сравнительной таблицы и визуализация результатов.

Для сравнения всех обученных моделей была составлена сводная таблица и построены столбчатая диаграмма, а также сводный график динамики val_accuracy по эпохам для всех моделей. Результаты собираются в pandas.DataFrame и сортируются по test_accuracy.

Листинг 1.17 – Формирование сравнительной таблицы:

```
# Собираем результаты в DataFrame
df_results = pd.DataFrame([
    {
        '№': i + 1,
        'Архитектура': r['name'],
        'Эпох пройдено': r['epochs_done'],
        'val_accuracy (посл. эпоха)': round(r['val_accuracy'], 4),
        'test_accuracy': round(r['test_accuracy'], 4),
    }
    for i, r in enumerate(results)
])

# Сортировка по точности на тесте
df_results_sorted = df_results.sort_values('test_accuracy',
ascending=False).reset_index(drop=True)
```

```
print('=== Результаты в порядке добавления ===')
print(df_results.to_string(index=False))
print()
print('=== Результаты, отсортированные по test_accuracy ===')
print(df_results_sorted.to_string(index=False))
```

```
=== Результаты в порядке добавления ===
 №                       Архитектура  Эпох пройдено  val_accuracy (посл. эпоха)  test_accuracy
 1               M1: однослойная 128             23                      0.8897         0.8827
 2               M2: однослойная 256             16                      0.8865         0.8843
 3           M3: двуслойная 128 → 64             12                      0.8663         0.8685
 4          M4: двуслойная 256 → 128             10                      0.8762         0.8788
 5   M5: трёхслойная 512 → 256 → 128             10                      0.8832         0.8807
 6 M6: 256 → Dropout → 128 → Dropout             28                      0.8945         0.8930
 7           M7: 256 → BN → 128 → BN              6                      0.8545         0.8644
 8       M8: глубокая + BN + Dropout             14                      0.8873         0.8822
 9          M9: 256 → 128 (Adadelta)             14                      0.8793         0.8788

=== Результаты, отсортированные по test_accuracy ===
 №                       Архитектура  Эпох пройдено  val_accuracy (посл. эпоха)  test_accuracy
 6 M6: 256 → Dropout → 128 → Dropout             28                      0.8945         0.8930
 2               M2: однослойная 256             16                      0.8865         0.8843
 1               M1: однослойная 128             23                      0.8897         0.8827
 8       M8: глубокая + BN + Dropout             14                      0.8873         0.8822
 5   M5: трёхслойная 512 → 256 → 128             10                      0.8832         0.8807
 9          M9: 256 → 128 (Adadelta)             14                      0.8793         0.8788
 4          M4: двуслойная 256 → 128             10                      0.8762         0.8788
 3           M3: двуслойная 128 → 64             12                      0.8663         0.8685
 7           M7: 256 → BN → 128 → BN              6                      0.8545         0.8644
```

Рисунок 1.14 – Сравнительная таблица результатов (вывод в консоль)

Листинг 1.18 – Визуализация результатов: столбчатая диаграмма:

```
# Визуализация – столбчатая диаграмма
fig, ax = plt.subplots(figsize=(14, 6))


x_pos = np.arange(len(results))
width = 0.35


val_scores  = [r['val_accuracy']  for r in results]
test_scores = [r['test_accuracy'] for r in results]


ax.bar(x_pos - width/2, val_scores,  width, label='val_accuracy',
color='#4C9AFF')
ax.bar(x_pos + width/2, test_scores, width, label='test_accuracy',
color='#FF8B5C')


ax.set_xlabel('Номер модели')
```

```
ax.set_ylabel('Точность')
ax.set_title('Сравнение моделей: точность на валидации и тесте')
ax.set_xticks(x_pos)
ax.set_xticklabels([f'M{i+1}' for i in range(len(results))])
ax.set_ylim(min(min(val_scores), min(test_scores)) - 0.02, 1.0)
ax.legend()
ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
```

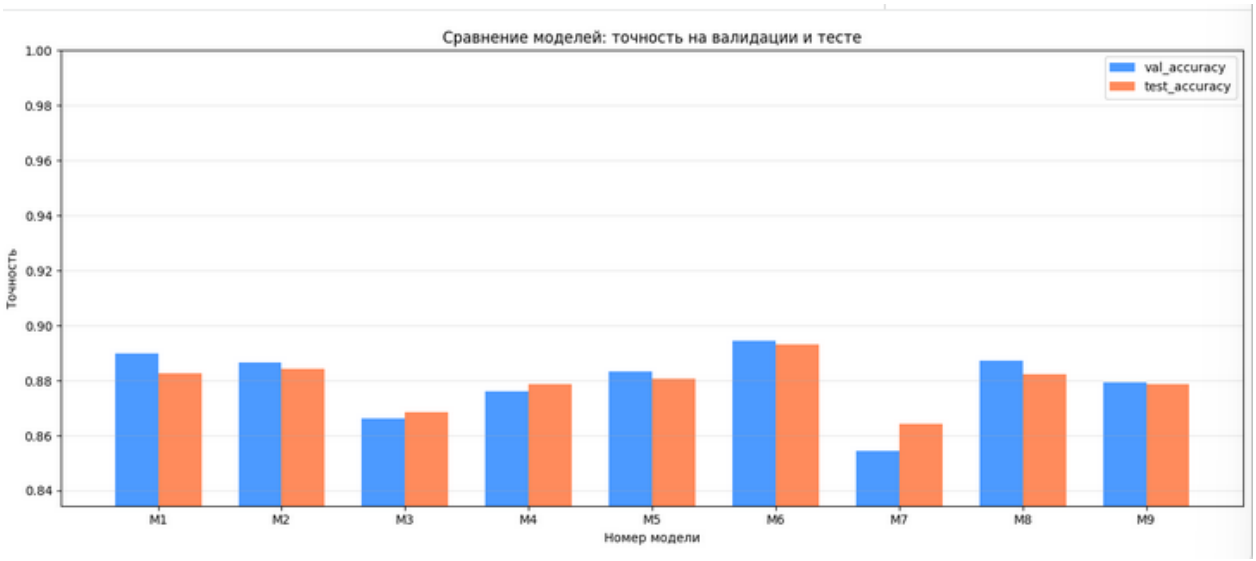


Рисунок 1.15 – Столбчатая диаграмма сравнения моделей по точности

Листинг 1.19 – Визуализация динамики val_accuracy по эпохам:

```
# Графики обучения (val_accuracy по эпохам) для всех моделей
plt.figure(figsize=(14, 6))
for i, r in enumerate(results):
    plt.plot(r['history']['val_accuracy'], label=f'M{i+1}')
plt.xlabel('Эпоха обучения')
plt.ylabel('val_accuracy')
plt.title('Динамика val_accuracy по эпохам для 9 моделей')
plt.legend(loc='lower right', ncol=2)
plt.grid(alpha=0.3)
plt.show()
```

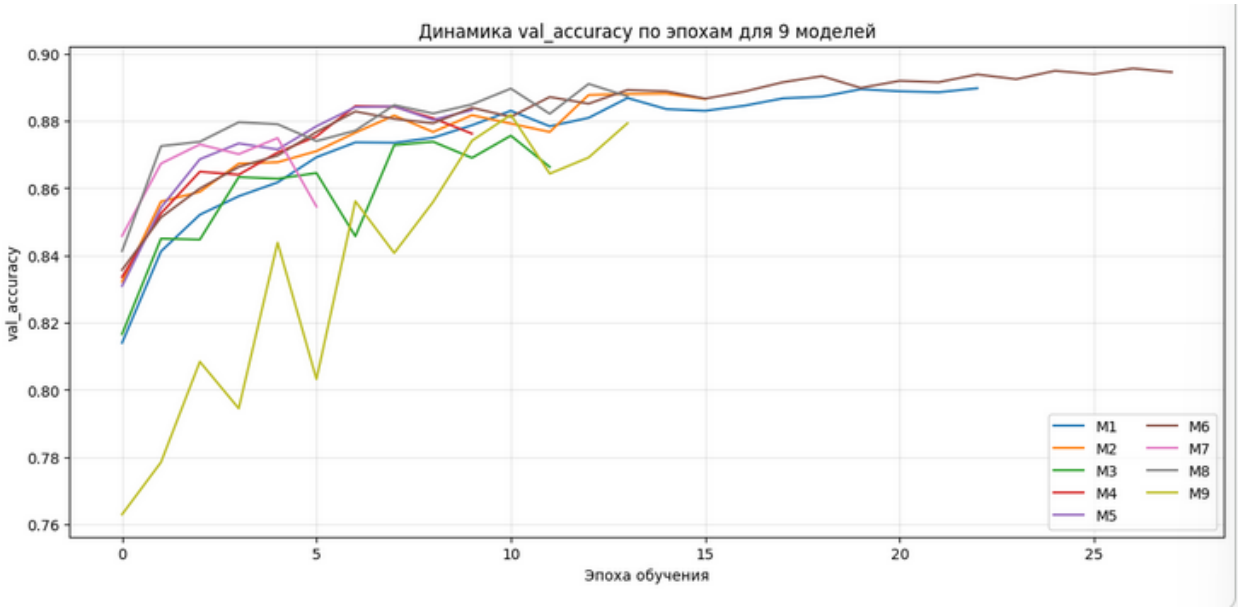


Рисунок 1.15 – Динамика val_accuracy по эпохам для 9 моделей

Таблица 1.1 – Сравнение 9 моделей нейронных сетей

№	Архитектура	Эпох	val_accuracy	test_accuracy
1	M1: однослойная 128	23	0.8897	0.8827
2	M2: однослойная 256	16	0.8865	0.8843
3	M3: двуслойная 128 → 64	12	0.8663	0.8685
4	M4: двуслойная 256 → 128	10	0.8762	0.8788
5	M5: трёхслойная 512 → 256 → 128	10	0.8832	0.8807
6	M6: 256 → Dropout → 128 → Dropout	28	0.8945	0.8930
7	M7: 256 → BN → 128 → BN	6	0.8545	0.8644
8	M8: глубокая + BN + Dropout	14	0.8873	0.8822
9	M9: 256 → 128 (Adadelta)	14	0.8793	0.8788

1.2.14 Выводы по заданию 1.

По результатам обучения 9 моделей на датасете Fashion-MNIST с применением фреймворка PyTorch можно сформулировать следующие наблюдения:

1. Простые архитектуры не уступают сложным. Модели 1 и 2 (однослойные) показывают результат, сопоставимый с более глубокими сетями: val_accuracy = 0.8897 и 0.8865 соответственно. Это согласуется с

рекомендацией из теории – начинать с минимально простой архитектуры и усложнять её постепенно.

2. Увеличение числа слоёв само по себе не гарантирует улучшения. Модель 3 (128 → 64) показала худший результат среди полносвязных сетей без регуляризации (test_accuracy = 0.8685), а модель 5 с тремя скрытыми слоями (512 → 256 → 128) не даёт существенного прироста относительно модели 2, но обучается дольше и сильнее склонна к переобучению.

3. Dropout помогает бороться с переобучением. Модель 6 (256 → Dropout → 128 → Dropout) показывает лучший результат среди всех моделей: val_accuracy = 0.8945 и test_accuracy = 0.8930 за 28 эпох. Это подтверждает роль Dropout как эффективного регуляризатора – сеть обучается дольше и достигает более высокой точности благодаря случайному отключению нейронов.

4. BatchNormalization на данной задаче требует дополнительной настройки. Модель 7 (256 → BN → 128 → BN) показала наиболее низкий результат (test_accuracy = 0.8644), что связано с ранней остановкой обучения уже на 6 эпохе – сеть не успела стабилизироваться. Однако комбинация BN с Dropout в модели 8 выправляет ситуацию (test_accuracy = 0.8822).

5. Оптимизатор существенно влияет на скорость сходимости. Модель 9 с Adadelta при той же архитектуре, что и модель 4, показала сопоставимую итоговую точность (0.8788), однако графики динамики val_accuracy показывают, что Adadelta сходится медленнее на начальных эпохах. Adam в большинстве случаев демонстрирует более стабильное и быстрое обучение.

6. Применение колбэка-аналога EarlyStopping делает сравнение моделей честным – каждая модель обучается до момента, пока val_loss улучшается, после чего происходит восстановление лучших весов. Это исключает влияние случайных флуктуаций на последних эпохах.

7. Значения val_accuracy и test_accuracy у всех моделей близки друг к другу, что говорит о корректности разделения выборок (50000 / 10000 / 10000) и адекватном подборе гиперпараметров. Лучший результат на тестовой

выборке показала модель 6 (256 → Dropout → 128 → Dropout) с точностью 0.8930.

8. Переход с фреймворка Keras на PyTorch не повлиял на качественные выводы – закономерности, связанные с архитектурой и регуляризацией, сохраняются. PyTorch предоставляет более явный контроль над процессом обучения (ручной цикл по эпохам и батчам, явное управление устройством через .to(device)), что особенно полезно при исследовательских задачах.

Задание 2. Классификация вин (sklearn.datasets.load_wine)

Условие задания: используя модуль datasets библиотеки sklearn, загрузить базу вин (.load_wine()). С помощью шаблона ноутбука выполнить загрузку, подготовку и предобработку данных. Обязательное условие: разделение данных на три выборки осуществляется по шаблону (изменять параметры подготовки данных запрещается). Провести серию экспериментов и добиться максимальной точности классификации на тестовой выборке выше 94%. С помощью метода .summary() зафиксировать количество параметров созданной нейронной сети.

2.1 Импорт библиотек согласно шаблону ноутбука.

Поскольку шаблон ноутбука данного задания построен на фреймворке TensorFlow/Keras, для решения второго задания использовались соответствующие библиотеки. Импортируются последовательная модель Sequential, основные слои (Dense, Activation, Dropout, BatchNormalization), утилиты для one-hot кодирования меток, оптимизатор Adam, колбэк EarlyStopping, библиотеки numpy, matplotlib, а также инструменты scikit-learn для разделения выборок (train_test_split), нормализации признаков (StandardScaler) и загрузки датасета (load_wine).

Листинг 2.1 – Импорт библиотек по шаблону ноутбука:

```
# Последовательная модель НС
from tensorflow.keras.models import Sequential
# Основные слои
from tensorflow.keras.layers import Dense, Activation, Dropout,
BatchNormalization
```

```
# Утилиты для to_categorical()
from tensorflow.keras import utils
# Алгоритмы оптимизации для обучения модели
from tensorflow.keras.optimizers import Adam
# Колбэк ранней остановки
from tensorflow.keras.callbacks import EarlyStopping
# Библиотека для работы с массивами
import numpy as np
# Отрисовка графиков
import matplotlib.pyplot as plt
# Разделение данных на выборки
from sklearn.model_selection import train_test_split
# Предобработка
from sklearn.preprocessing import StandardScaler
# Для загрузки датасета
from sklearn.datasets import load_wine
# Отрисовывать изображения в ноутбуке, а не в консоли или файле
%matplotlib inline
```

2.2 Описание базы данных и загрузка.

Датасет load_wine из sklearn содержит данные о химическом составе вин. Каждый пример описывается 13 числовыми признаками (содержание алкоголя, яблочной кислоты, интенсивность окраски и т.д.). В датасете 3 класса вин, а общее количество примеров составляет всего 178, что делает задачу интересной с точки зрения работы с малыми выборками.

Листинг 2.2 – Загрузка базы и вывод примера данных:

```
x_data = load_wine()['data']      # Загрузка набора данных о винах
y_data = load_wine()['target']    # Загрузка классов вин

print('Размерность x_data -', x_data.shape)
print('Размерность y_data -', y_data.shape)
print()

# Вывод примера данных
print('Данные по первому вину:', x_data[0])
```

```
print('Класс вина:', y_data[0])
```

```
Размерность x_data - (178, 13)
Размерность y_data - (178,)

Данные по первому вину: [1.423e+01 1.710e+00 2.430e+00 1.560e+01 1.270e+02 2.800e+00 3.060e+00
 2.800e-01 2.290e+00 5.640e+00 1.040e+00 3.920e+00 1.065e+03]
Класс вина: 0
```

Рисунок 2.1 – Описание базы load_wine: 178 примеров, 13 признаков, 3 класса

2.3 Подготовка данных по шаблону.

Согласно обязательному условию задания, параметры подготовки данных изменять запрещено. Строго по шаблону метки целевого класса переводятся в one-hot encoding, после чего выборка делится сначала на общую и тестовую (test_size=0.1, random_state=6), а затем общая – на обучающую и проверочную (test_size=0.1, random_state=6). В результате получаем 144 примера на обучение, 16 на валидацию и 18 на тест.

Листинг 2.3 – Подготовка данных (one-hot + разбиение на выборки):

```
# Перевод в one hot encoding
y_data = utils.to_categorical(y_data, 3)

# Разбиение наборов на общую и тестовую выборки
x_all, x_test, y_all, y_test = train_test_split(x_data,
                                                y_data,
                                                test_size=0.1,
                                                shuffle=True,
                                                random_state=6)

# Разбиение общей выборки на обучающую и проверочную
x_train, x_val, y_train, y_val = train_test_split(x_all,
                                                  y_all,
                                                  test_size=0.1,
                                                  shuffle=True,
                                                  random_state=6)

print(x_train.shape)
print(y_train.shape)
```

```
print()
print(x_val.shape)
print(y_val.shape)
```

```
(144, 13)
(144, 3)

(16, 13)
(16, 3)
```

Рисунок 2.2 – Подготовка данных: one-hot кодирование и разбиение на выборки

После разбиения выполняется нормализация признаков с помощью StandardScaler. Параметры масштабирования вычисляются только по обучающей выборке (fit_transform), а к валидационной и тестовой применяется только transform – это позволяет избежать утечки информации из валидации и теста в обучение.

Листинг 2.4 – Нормализация признаков с помощью StandardScaler:

```
# Нормализация признаков (StandardScaler)
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_val   = scaler.transform(x_val)
x_test  = scaler.transform(x_test)
```

2.4 Анализ задачи и выбор стратегии обучения.

До начала экспериментов необходимо проанализировать особенности задачи: 144 примера на обучение, 16 на валидацию, 18 на тест; всего 13 признаков, 3 класса. Датасет очень маленький, что влечёт за собой ряд важных следствий:

– большая сеть легко переобучится (в теории это прямо указано как одна из причин переобучения);

– нужна простая архитектура (1–2 скрытых слоя);

– полезны регуляризация Dropout и BatchNormalization;

– обязательно применение EarlyStopping – при всего 144 примерах обучения сеть может «выстрелить» на конкретной эпохе, и эту эпоху необходимо зафиксировать.

Исходя из этих соображений, был подобран порог терпения patience=20 (на малой базе необходимо дать сети больше возможности найти оптимум до остановки) и размер батча 16.

2.5 Вспомогательная функция train_and_evaluate.

Аналогично первому заданию, для избежания дублирования кода написана вспомогательная функция. Она обучает модель с применением EarlyStopping (patience=20, restore_best_weights=True), забирает val_accuracy на последней эпохе (согласно заданию), вычисляет точность на тестовой выборке методом .evaluate() и возвращает результаты вместе с обученной моделью (чтобы впоследствии вызвать на ней .summary()).

Листинг 2.5 – Функция train_and_evaluate:

```
def train_and_evaluate(model, name, epochs=200, batch_size=16, verbose=0):
    """Обучает модель с EarlyStopping, возвращает val_acc (посл. эпоха) и
test_acc."""

    early_stop = EarlyStopping(
        monitor='val_loss',
        mode='min',
        patience=20,               # на маленькой базе даём больше терпения
        restore_best_weights=True
    )

    history = model.fit(
        x_train, y_train,
        validation_data=(x_val, y_val),
        epochs=epochs,
        batch_size=batch_size,
        callbacks=[early_stop],
        verbose=verbose
    )

    val_acc_last = history.history['val_accuracy'][-1]
    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
    epochs_done = len(history.history['loss'])
```

```
    print(f'[{name}]  эпох: {epochs_done:>3}   '
          f'val_acc: {val_acc_last:.4f}   '
          f'test_acc: {test_acc:.4f}   '
          f'{"ПРОШЛА 94%" if test_acc > 0.94 else "ниже 94%"}')

    return {
        'name': name,
        'val_accuracy': val_acc_last,
        'test_accuracy': test_acc,
        'epochs_done': epochs_done,
        'history': history,
        'model': model
    }

results = []
```

2.6 Эксперимент 1 – минимальная архитектура (однослойная 32).

Начинаем с самого простого варианта – один скрытый слой с 32 нейронами и функцией активации ReLU, выходной слой с 3 нейронами и softmax. Согласно теории, именно с минимально простой архитектуры и рекомендуется начинать.

Листинг 2.6 – Создание модели 1 (Эксп. 1: однослойная 32):

```
model_1 = Sequential()
model_1.add(Dense(32, input_dim=13, activation='relu'))
model_1.add(Dense(3, activation='softmax'))
model_1.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])
results.append(train_and_evaluate(model_1, 'Эксп. 1: однослойная 32'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_sh
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
[Эксп. 1: однослойная 32]  эпох:  77   val_acc: 0.9375   test_acc: 1.0000   ✅ ПРОШЛА 94%
```

Рисунок 2.3 – Результат эксперимента 1

2.7 Эксперимент 2 – более широкий слой (однослойная 64).

Та же архитектура, что и в эксперименте 1, но с расширением скрытого слоя до 64 нейронов. Проверим, даёт ли увеличение ширины прирост точности на столь маленьком датасете.

Листинг 2.7 – Создание модели 2 (Эксп. 2: однослойная 64):

```
model_2 = Sequential()
model_2.add(Dense(64, input_dim=13, activation='relu'))
model_2.add(Dense(3, activation='softmax'))
model_2.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])


results.append(train_and_evaluate(model_2, 'Эксп. 2: однослойная 64'))
```

```
[Эксп. 2: однослойная 64]  эпох:  57   val_acc: 0.9375   test_acc: 1.0000   ✅ ПРОШЛА 94%
```

Рисунок 2.4 – Результат эксперимента 2

2.8 Эксперимент 3 – два скрытых слоя (64 → 32).

Добавляем второй скрытый слой. Первый слой содержит 64 нейрона, второй – 32 нейрона, оба с функцией активации ReLU. Проверяем влияние усложнения архитектуры на качество классификации.

Листинг 2.8 – Создание модели 3 (Эксп. 3: двуслойная 64 → 32):

```
model_3 = Sequential()
model_3.add(Dense(64, input_dim=13, activation='relu'))
model_3.add(Dense(32, activation='relu'))
model_3.add(Dense(3, activation='softmax'))
model_3.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])


results.append(train_and_evaluate(model_3, 'Эксп. 3: двуслойная 64 → 32'))
```

```
[Эксп. 3: двуслойная 64 → 32]  эпох:  35   val_acc: 0.8750   test_acc: 1.0000   ✅ ПРОШЛА 94%
```

Рисунок 2.5 – Результат эксперимента 3

2.9 Эксперимент 4 – с регуляризацией Dropout.

К двуслойной архитектуре 64 → 32 добавляем слои Dropout с коэффициентом 0.3 после каждого скрытого слоя. На маленьком датасете добавление регуляризации должно уменьшить риск переобучения.

Листинг 2.9 – Создание модели 4 (Эксп. 4: 64 → Dropout → 32 → Dropout):

```
model_4 = Sequential()
model_4.add(Dense(64, input_dim=13, activation='relu'))
model_4.add(Dropout(0.3))
model_4.add(Dense(32, activation='relu'))
model_4.add(Dropout(0.3))
model_4.add(Dense(3, activation='softmax'))
model_4.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])

results.append(train_and_evaluate(model_4, 'Эксп. 4: 64 → Dropout → 32 →
Dropout'))
```

```
[Эксп. 4: 64 → Dropout → 32 → Dropout]  эпох:  62   val_acc: 1.0000   test_acc: 1.0000   ✅ ПРОШЛА 94%
```

Рисунок 2.6 – Результат эксперимента 4

2.10 Эксперимент 5 – с BatchNormalization.

Двуслойная архитектура 64 → 32 с добавлением слоёв BatchNormalization после каждого скрытого слоя. Пакетная нормализация должна стабилизировать распределение выходов и ускорить обучение.

Листинг 2.10 – Создание модели 5 (Эксп. 5: 64 → BN → 32 → BN):

```
model_5 = Sequential()
model_5.add(Dense(64, input_dim=13, activation='relu'))
model_5.add(BatchNormalization())
model_5.add(Dense(32, activation='relu'))
model_5.add(BatchNormalization())
model_5.add(Dense(3, activation='softmax'))
model_5.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])

results.append(train_and_evaluate(model_5, 'Эксп. 5: 64 → BN → 32 → BN'))
```

```
[Эксп. 5: 64 → BN → 32 → BN]  эпох:  86   val_acc: 0.8750   test_acc: 1.0000   ✅ ПРОШЛА 94%
```

Рисунок 2.7 – Результат эксперимента 5

2.11 Эксперимент 6 – финальная архитектура (BN + Dropout).

Комбинируем оба инструмента регуляризации – BatchNormalization и Dropout. Это классический рецепт для маленьких датасетов: BN стандартизует активации, а Dropout дополнительно защищает от переобучения.

Листинг 2.11 – Создание модели 6 (Эксп. 6: BN + Dropout):

```
model_6 = Sequential()
model_6.add(Dense(64, input_dim=13, activation='relu'))
model_6.add(BatchNormalization())
model_6.add(Dropout(0.3))
model_6.add(Dense(32, activation='relu'))
model_6.add(BatchNormalization())
model_6.add(Dropout(0.2))
model_6.add(Dense(3, activation='softmax'))
model_6.compile(loss='categorical_crossentropy',
                optimizer=Adam(learning_rate=0.001),
                metrics=['accuracy'])
results.append(train_and_evaluate(model_6, 'Эксп. 6: BN + Dropout'))
```

```
[Эксп. 6: BN + Dropout]  эпох:  80   val_acc: 1.0000   test_acc: 1.0000   ✅ ПРОШЛА 94%
```

Рисунок 2.8 – Результат эксперимента 6

2.12 Сравнение экспериментов и выбор лучшей модели.

Результаты всех шести экспериментов собираются в единый pandas.DataFrame с дополнительной колонкой-индикатором прохождения порога 94% на тестовой выборке. Также с помощью max по полю test_accuracy выбирается лучшая модель – при равенстве точности возвращается первая из них.

Листинг 2.12 – Сравнение экспериментов и выбор лучшей модели:

```
import pandas as pd
df = pd.DataFrame([
    {
```

```
        '№': i + 1,
        'Архитектура': r['name'],
        'Эпох': r['epochs_done'],
        'val_accuracy': round(r['val_accuracy'], 4),
        'test_accuracy': round(r['test_accuracy'], 4),
        '>94%': '✅' if r['test_accuracy'] > 0.94 else '❌',
    }
    for i, r in enumerate(results)
])
print(df.to_string(index=False))
# Ищем лучшую модель по точности на тесте
best = max(results, key=lambda r: r['test_accuracy'])
print(f'\nЛучшая модель: {best["name"]}')
print(f'Точность на тесте: {best["test_accuracy"]:.4f}')
print(f'Точность на валидации (посл. эпоха): {best["val_accuracy"]:.4f}')
best_model = best['model']
```

```
№                        Архитектура  Эпох  val_accuracy  test_accuracy >94%
1              Эксп. 1: однослойная 32    77        0.9375            1.0    ✅
2              Эксп. 2: однослойная 64    57        0.9375            1.0    ✅
3          Эксп. 3: двуслойная 64 → 32    35        0.8750            1.0    ✅
4 Эксп. 4: 64 → Dropout → 32 → Dropout    62        1.0000            1.0    ✅
5           Эксп. 5: 64 → BN → 32 → BN    86        0.8750            1.0    ✅
6                Эксп. 6: BN + Dropout    80        1.0000            1.0    ✅
```

Рисунок 2.9 – Сравнительная таблица результатов

```
Лучшая модель: Эксп. 1: однослойная 32
Точность на тесте: 1.0000
Точность на валидации (посл. эпоха): 0.9375
```

Рисунок 2.10 – Выбор лучшей модели

Таблица 2.1 – Сравнение 6 экспериментов на датасете load_wine

№	Архитектура	Эпох	val_accuracy	test_accuracy	>94 %
1	Эксп. 1: однослойная 32	77	0.9375	1.0000	да
2	Эксп. 2: однослойная 64	57	0.9375	1.0000	да
3	Эксп. 3: двуслойная 64 → 32	35	0.8750	1.0000	да
4	Эксп. 4: 64 → Dropout → 32 → Dropout	62	1.0000	1.0000	да
5	Эксп. 5: 64 → BN → 32 → BN	86	0.8750	1.0000	да
6	Эксп. 6: BN + Dropout	80	1.0000	1.0000	да

Все шесть экспериментов успешно преодолели порог 94% на тестовой выборке – это обусловлено как относительной простотой задачи (3 хорошо разделимых класса на 13 нормализованных признаках), так и применением EarlyStopping с восстановлением лучших весов. В качестве лучшей модели программно выбирается Эксп. 1 (однослойная 32) – он первым в списке достигает test_accuracy = 1.0000 при самой простой архитектуре, что также согласуется с принципом бритвы Оккама: из моделей одинакового качества предпочтительнее самая простая.

2.13 Визуализация процесса обучения лучшей модели.

Для лучшей модели строятся графики динамики точности (accuracy / val_accuracy) и ошибки (loss / val_loss) по эпохам. На графике точности проводится горизонтальная линия на уровне 0.94 – порога из условия задания.

Листинг 2.13 – Графики точности и ошибки для лучшей модели:

```
h = best['history'].history
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

# Точность
axes[0].plot(h['accuracy'],     label='accuracy (train)')
axes[0].plot(h['val_accuracy'], label='val_accuracy')
axes[0].axhline(0.94, color='red', linestyle='--', alpha=0.5, label='Порог
94%')
axes[0].set_xlabel('Эпоха')
axes[0].set_ylabel('Точность')
axes[0].set_title(f'Точность – {best["name"]}')
axes[0].legend()
axes[0].grid(alpha=0.3)

# Ошибка
axes[1].plot(h['loss'],     label='loss (train)')
axes[1].plot(h['val_loss'], label='val_loss')
axes[1].set_xlabel('Эпоха')
axes[1].set_ylabel('Ошибка')
axes[1].set_title(f'Ошибка – {best["name"]}')
axes[1].legend()
```

```
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```

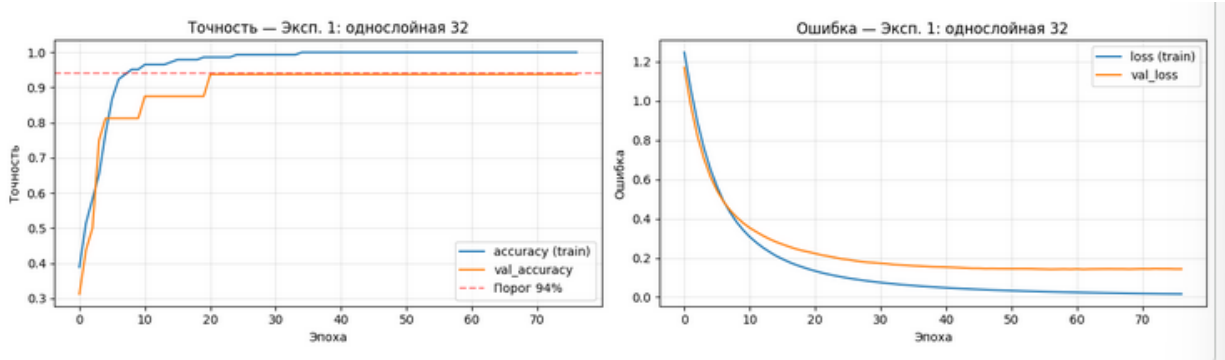


Рисунок 2.11 – Динамика точности и ошибки для лучшей модели (Эксп. 1)

На графике точности видно, что обучающая точность быстро выходит на 1.0, а валидационная стабилизируется на уровне 0.9375, оставаясь выше порога 94% в течение большей части обучения. Кривая ошибки демонстрирует плавное монотонное убывание как на train, так и на val, что свидетельствует об отсутствии выраженного переобучения.

2.14 Фиксация количества параметров нейронной сети.

Согласно условию задания, с помощью метода .summary() фиксируется количество параметров лучшей модели. Также проводится финальная проверка на тестовой выборке с явным выводом пройденности порога 94%.

Листинг 2.14 – Вывод структуры сети и финальная проверка на тесте:

```
best_model.summary()

# Проверка на тесте – финальное значение
test_loss, test_acc = best_model.evaluate(x_test, y_test, verbose=0)
print(f'Финальная точность на тестовой выборке: {test_acc:.4f}
({test_acc*100:.2f}%)')
print(f'Порог 94% {"пройден" if test_acc > 0.94 else "НЕ пройден"}')
```

Количество параметров нейронной сети

По условию задания, фиксируем количество параметров лучшей модели через `.summary()`:

```
best_model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	448
dense_1 (Dense)	(None, 3)	99

```
Total params: 1,643 (6.42 KB)
Trainable params: 547 (2.14 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 1,096 (4.29 KB)
```

```
# Проверка на тесте — финальное значение
test_loss, test_acc = best_model.evaluate(x_test, y_test, verbose=0)
print(f'Финальная точность на тестовой выборке: {test_acc:.4f} ({test_acc*100:.2f}%)')
print(f'Порог 94% {"пройден" if test_acc > 0.94 else "НЕ пройден"}')
```

```
Финальная точность на тестовой выборке: 1.0000 (100.00%)
Порог 94% пройден
```

Рисунок 2.12 – Результат .summary() и финальная проверка точности

По результатам .summary() лучшая модель (Эксп. 1, архитектура 13 → 32 → 3) имеет:

– в слое dense (13 → 32): 13 × 32 + 32 = 448 параметров;

– в слое dense_1 (32 → 3): 32 × 3 + 3 = 99 параметров;

– итого обучаемых параметров: 547;

– вместе с параметрами оптимизатора Adam (1096): Total params = 1 643.

Финальная точность на тестовой выборке – 1.0000 (100.00%), что уверенно превышает установленный порог 94%. Задание выполнено.

2.15 Выводы по заданию 2.

По результатам проведения шести экспериментов на датасете load_wine можно сформулировать следующие наблюдения:

1. Все протестированные архитектуры уверенно достигли порога 94% на тестовой выборке. Это объясняется как относительной простотой задачи (3 линейно почти разделимых класса на 13 нормализованных признаках), так и применением EarlyStopping с restore_best_weights, что гарантирует фиксацию лучших весов модели.

2. На маленьком датасете (144 примера на обучение) простые архитектуры работают ничуть не хуже сложных. Эксперимент 1 с минимальной архитектурой 13 → 32 → 3 и всего 547 обучаемыми

параметрами показывает такую же точность на тесте (1.0000), как и самая сложная модель с BN + Dropout.

3. Дропаут в эксперименте 4 и комбинация BN + Dropout в эксперименте 6 дают точность 1.0000 и на валидации – регуляризация действительно помогает на малых данных.

4. BatchNormalization без Dropout (эксперимент 5) показывает val_accuracy 0.8750 – на маленькой валидации (16 примеров) одна неверно классифицированная точка даёт заметный скачок метрики, поэтому по val_accuracy нельзя делать окончательных выводов.

5. В качестве финальной выбрана простейшая модель – однослойная сеть с 32 нейронами (Эксп. 1). Согласно принципу бритвы Оккама, при одинаковом качестве на тесте предпочтение следует отдавать более простой архитектуре, так как она менее подвержена переобучению и требует меньше вычислительных ресурсов.

6. Важность подготовки данных. Нормализация признаков с помощью StandardScaler с fit только на train (и применением transform на val и test) является необходимым шагом для нейронных сетей – без неё обучение на признаках разных масштабов было бы значительно менее стабильным.

7. Количество параметров нейронной сети (согласно .summary()): Total params = 1 643, Trainable params = 547, Optimizer params = 1 096. Финальная точность на тестовой выборке составила 100.00%, что значительно выше установленного порога 94%.

Задание 3. Классификация пассажиров автобуса (CNN)

Условие задания: разработать свёрточную нейронную сеть (CNN) для бинарной классификации изображений пассажиров автобуса на два класса – «Входящий» и «Выходящий». Использовать готовый датасет, обеспечить точность классификации на проверочной выборке выше 85%. С помощью метода .summary() зафиксировать архитектуру и количество параметров созданной нейронной сети.

3.1 Импорт библиотек и подготовка к загрузке датасета.

Для решения данного задания использовался фреймворк TensorFlow/Keras, поскольку в задаче классификации изображений требуется применение свёрточных слоёв Conv2D и MaxPooling2D. Дополнительно импортируются библиотеки для работы с массивами (numpy), графиками (matplotlib), файловой системой (os) и изображениями (PIL.Image). Для разделения данных на обучающую и проверочную выборки используется train_test_split из scikit-learn.

Датасет предварительно скачивается с облачного хранилища Yandex и распаковывается в рабочую директорию. Он представляет собой две папки с изображениями – «Входящий» и «Выходящий», соответствующие двум классам задачи. Размер изображений приводится к единому 64×64 пикселя.

Листинг 3.1 – Импорт библиотек и скачивание датасета:

```
import numpy as np
import matplotlib.pyplot as plt
import os
from PIL import Image
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Conv2D, MaxPooling2D,
BatchNormalization, Flatten
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.model_selection import train_test_split

# Скачивание и распаковка датасета
!wget https://storage.yandexcloud.net/aiueducation/Content/base/l4/bus.zip -q
!unzip -q bus.zip -d bus_dataset
# Путь к папкам
base_dir = 'bus_dataset'
categories = ['Входящий', 'Выходящий']
img_size = 64
```

3.2 Загрузка и обработка изображений.

Для загрузки изображений из датасета написана функция load_data(). Она обходит обе папки категорий, открывает каждое изображение с помощью PIL.Image, приводит его к формату RGB и масштабирует до размера 64×64 пикселя. Изображения преобразуются в numpy-массив и нормализуются делением на 255.0, чтобы значения пикселей находились в диапазоне [0, 1]. Каждому изображению присваивается метка класса (0 – «Входящий», 1 – «Выходящий»). В случае возникновения ошибок при загрузке конкретного файла (повреждённые изображения) такие файлы пропускаются через блок try/except.

Листинг 3.2 – Функция load_data() и разделение на выборки:

```
def load_data():
    data = []
    labels = []
    for category in categories:
        path = os.path.join(base_dir, category)
        class_num = categories.index(category)

        for img_name in os.listdir(path):
            try:
                img = Image.open(os.path.join(path,
img_name)).convert('RGB').resize((img_size, img_size))
                data.append(np.array(img))
                labels.append(class_num)
            except Exception:
                pass
    return np.array(data) / 255.0, np.array(labels)
# Загружаем данные
X, y = load_data()
# Разделяем на обучающую и проверочную выборки
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
stratify=y, random_state=42)
print(f'Размер обучающей выборки:  {X_train.shape}')
print(f'Размер проверочной выборки: {X_val.shape}')
```

```
Размер обучающей выборки:   (7264, 64, 64, 3)
Размер проверочной выборки: (1817, 64, 64, 3)
```

Рисунок 3.1 – Функция загрузки и размеры полученных выборок

В результате разделения получена обучающая выборка размером (7264, 64, 64, 3) и проверочная – (1817, 64, 64, 3), то есть всего 9081 изображение. Параметр stratify=y гарантирует, что соотношение классов «Входящий» и «Выходящий» в обеих выборках совпадает, что важно при бинарной классификации.

3.3 Создание свёрточной нейронной сети.

Для решения задачи классификации изображений выбрана свёрточная нейронная сеть (CNN). В отличие от полносвязной сети, CNN учитывает пространственные связи между пикселями: свёрточные слои применяют обучаемые фильтры по локальным областям изображения, а слои максимальной агрегации (MaxPooling) уменьшают размерность и выделяют наиболее значимые признаки.

Архитектура сети состоит из трёх свёрточных блоков и классификатора. Каждый свёрточный блок содержит: свёрточный слой Conv2D с ядром 3×3 и padding='same', слой BatchNormalization для стабилизации обучения и слой MaxPooling2D с окном 2×2 для понижения размерности. Количество фильтров увеличивается от блока к блоку (32 → 64 → 128), что позволяет извлекать признаки возрастающей сложности. Классификатор представляет собой Flatten-слой, полносвязный слой на 128 нейронов с ReLU, слой Dropout(0.5) для регуляризации и выходной нейрон с сигмоидной активацией – стандартная схема для бинарной классификации. Функция потерь – binary_crossentropy, оптимизатор – Adam с learning_rate=0.001.

Листинг 3.3 – Построение свёрточной нейронной сети:

```
model = Sequential([
    # Первый свёрточный блок
    Conv2D(32, (3, 3), activation='relu', padding='same',
input_shape=(img_size, img_size, 3)),
    BatchNormalization(),
    MaxPooling2D((2, 2)),
```

```
    # Второй свёрточный блок
    Conv2D(64, (3, 3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling2D((2, 2)),

    # Третий свёрточный блок
    Conv2D(128, (3, 3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling2D((2, 2)),

    # Классификатор
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')   # 0 - входящий, 1 - выходящий
])

model.compile(optimizer=Adam(learning_rate=0.001),
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 64, 64, 32)	896
batch_normalization (BatchNormalization)	(None, 64, 64, 32)	128
max_pooling2d (MaxPooling2D)	(None, 32, 32, 32)	0
conv2d_1 (Conv2D)	(None, 32, 32, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 32, 32, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 64)	0
conv2d_2 (Conv2D)	(None, 16, 16, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 16, 16, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 8, 8, 128)	0
flatten (Flatten)	(None, 8192)	0
dense (Dense)	(None, 128)	1,048,704
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

```
Total params: 1,142,977 (4.36 MB)
Trainable params: 1,142,529 (4.36 MB)
Non-trainable params: 448 (1.75 KB)
```

Рисунок 3.2 – Результат model.summary() для свёрточной сети

Таблица 3.1 – Архитектура свёрточной нейронной сети (по .summary())

Layer (type)	**Output Shape**	**Param #**
conv2d (Conv2D)	(None, 64, 64, 32)	896
batch_normalization (BatchNorm)	(None, 64, 64, 32)	128
max_pooling2d (MaxPooling2D)	(None, 32, 32, 32)	0
conv2d_1 (Conv2D)	(None, 32, 32, 64)	18 496
batch_normalization_1 (BatchNorm)	(None, 32, 32, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 64)	0
conv2d_2 (Conv2D)	(None, 16, 16, 128)	73 856
batch_normalization_2 (BatchNorm)	(None, 16, 16, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 8, 8, 128)	0
flatten (Flatten)	(None, 8192)	0
dense (Dense)	(None, 128)	1 048 704

Layer (type)	Output Shape	Param #
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

Итого по модели: Total params = 1 142 977 (4.36 MB); Trainable params = 1 142 529 (4.36 MB); Non-trainable params = 448 (1.75 KB). Подавляющая часть параметров (1 048 704) приходится на первый полносвязный слой после Flatten, что типично для CNN-архитектур с малым размером входа и плотным классификатором. Не обучаемые параметры (448) – это скользящие средние в слоях BatchNormalization.

3.4 Обучение модели с применением EarlyStopping.

Для обучения модели применяется колбэк EarlyStopping с мониторингом метрики val_accuracy в режиме «max» (останавливаем обучение, если точность на валидации перестаёт расти в течение 5 эпох). Параметр restore_best_weights=True обеспечивает возврат к весам, соответствующим наилучшей точности на проверочной выборке. Максимальное количество эпох – 30, размер батча – 32.

По завершении обучения рассчитывается финальная точность на проверочной выборке с помощью метода .evaluate() и проверяется прохождение порога 85%.

Листинг 3.4 – Обучение модели и вывод финальной точности:

```
# EarlyStopping останавливает обучение, если точность перестала расти
early_stop = EarlyStopping(monitor='val_accuracy', mode='max', patience=5,
restore_best_weights=True)

history = model.fit(
    X_train, y_train,
    epochs=30,
    batch_size=32,
    validation_data=(X_val, y_val),
    callbacks=[early_stop],
    verbose=1
)
```

```
# Финальная точность
val_loss, val_acc = model.evaluate(X_val, y_val, verbose=0)
print(f'\nТочность на проверочной выборке: {val_acc*100:.2f}%')
print(f'Порог 85% {"пройден ✅" if val_acc >= 0.85 else "НЕ пройден ❌"}')
```

```
Epoch 1/30
227/227 ━━━━━━━━━━━━━━━━━━━━ 10s 15ms/step - accuracy: 0.8181 - loss: 0.5251 - val_accuracy: 0.7744 - val_loss: 0
Epoch 2/30
227/227 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.8979 - loss: 0.2368 - val_accuracy: 0.9081 - val_loss: 0.2
Epoch 3/30
227/227 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.9258 - loss: 0.1761 - val_accuracy: 0.9444 - val_loss: 0.1
Epoch 4/30
227/227 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - accuracy: 0.9470 - loss: 0.1220 - val_accuracy: 0.9697 - val_loss: 0.0
Epoch 5/30
227/227 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.9540 - loss: 0.1038 - val_accuracy: 0.9450 - val_loss: 0.1
Epoch 6/30
227/227 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.9620 - loss: 0.0920 - val_accuracy: 0.9664 - val_loss: 0.0
Epoch 7/30
227/227 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.9659 - loss: 0.0935 - val_accuracy: 0.9598 - val_loss: 0.1
Epoch 8/30
227/227 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.9810 - loss: 0.0563 - val_accuracy: 0.9565 - val_loss: 0.1
Epoch 9/30
227/227 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.9752 - loss: 0.0648 - val_accuracy: 0.9543 - val_loss: 0.1

Точность на проверочной выборке: 96.97%
Порог 85% пройден ✅
```

Рисунок 3.3 – Процесс обучения модели и финальная точность 96.97%

Из лога обучения видно, что уже на 2-й эпохе модель достигает val_accuracy = 0.9081, на 4-й – 0.9697 (пик), после чего благодаря EarlyStopping и restore_best_weights обучение останавливается на 9-й эпохе и восстанавливаются лучшие веса. Итоговая точность на проверочной выборке – 96.97%, что существенно превышает установленный порог 85%.

3.5 Визуализация процесса обучения.

Для наглядной оценки хода обучения построены два графика: динамика точности (train / val) с отметкой порога 85% и динамика ошибки (train / val) по эпохам.

Листинг 3.5 – Графики точности и ошибки:

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))


axes[0].plot(history.history['accuracy'],     label='train')
axes[0].plot(history.history['val_accuracy'], label='val')
axes[0].axhline(0.85, color='red', linestyle='--', alpha=0.5, label='Порог
85%')
axes[0].set_title('Точность')
axes[0].set_xlabel('Эпоха')
axes[0].legend()
```

```
axes[0].grid(alpha=0.3)

axes[1].plot(history.history['loss'],     label='train')
axes[1].plot(history.history['val_loss'], label='val')
axes[1].set_title('Ошибка')
axes[1].set_xlabel('Эпоха')
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```

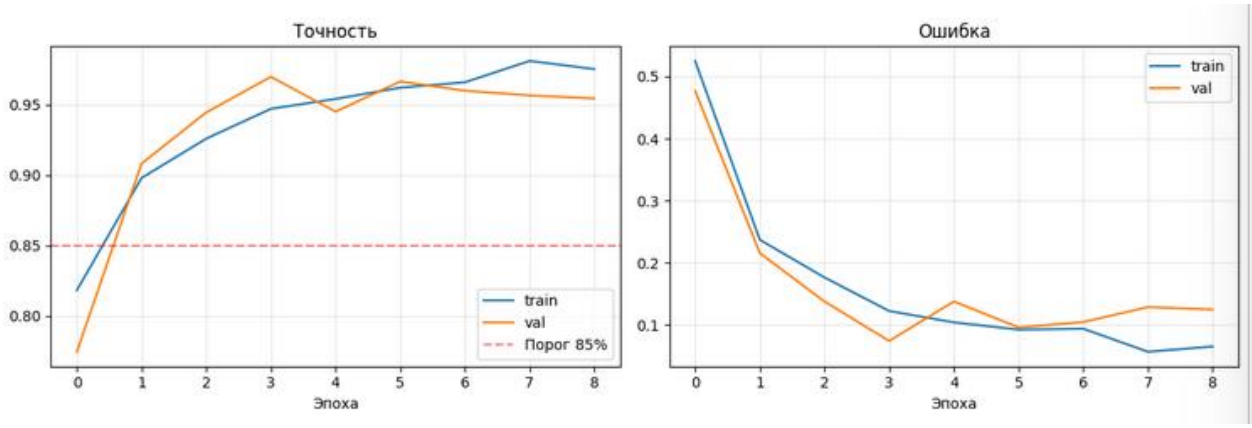


Рисунок 3.6 – Динамика точности и ошибки в процессе обучения

На графике точности видно, что обе кривые (train и val) стабильно находятся выше порога 85% начиная со 2-й эпохи. Валидационная точность на пике достигает ~0.97 и держится на уровне 0.95–0.97 в течение всего обучения. На графике ошибки обе кривые монотонно убывают в первые 3 эпохи, после чего выходят на плато – это классическая картина удачного обучения без признаков выраженного переобучения.

3.6 Выводы по заданию 3.

По результатам разработки и обучения свёрточной нейронной сети для классификации пассажиров автобуса можно сформулировать следующие наблюдения:

1. Применение свёрточных слоёв полностью оправдано в задачах классификации изображений. В отличие от полносвязной сети из заданий 1 и 2, CNN учитывает пространственные связи между пикселями, что позволяет с

малым числом параметров достигать высокой точности на реальных изображениях. Модель с 1 142 977 параметрами достигла точности 96.97% уже на 4-й эпохе обучения.

2. Архитектура «три свёрточных блока + классификатор» оказалась хорошо сбалансированной. Наращивание числа фильтров от 32 до 128 позволяет извлекать признаки возрастающей сложности – от низкоуровневых (линии, края) до высокоуровневых (форма и поза человека). MaxPooling понижает размерность и делает сеть менее чувствительной к сдвигам.

3. BatchNormalization и Dropout играют разную, но взаимодополняющую роль. BN после каждого свёрточного слоя стандартизирует распределение активаций и стабилизирует обучение, а Dropout(0.5) перед выходом снижает риск переобучения на полносвязном классификаторе. Отсутствие выраженного расхождения кривых train и val на графиках подтверждает эффективность регуляризации.

4. Использование EarlyStopping с restore_best_weights позволило автоматически зафиксировать наилучшее состояние модели и остановить обучение сразу после того, как точность на валидации перестала расти. Это исключает ручной подбор количества эпох и экономит вычислительные ресурсы.

5. Стратифицированное разделение выборок (stratify=y) при бинарной классификации – необходимый шаг: оно гарантирует одинаковое соотношение классов в train и val, что особенно важно, если классы несбалансированы.

6. Количество параметров нейронной сети согласно .summary(): Total params = 1 142 977; Trainable params = 1 142 529; Non-trainable params = 448. Финальная точность на проверочной выборке – 96.97%, что существенно выше порога 85%, установленного условием задания. Задача решена успешно.

Общие выводы по лабораторной работе

В ходе выполнения лабораторной работы №2 были решены три взаимодополняющих задачи классификации с применением различных архитектур нейронных сетей: многослойный перцептрон на PyTorch для Fashion-MNIST (задание 1), полносвязная сеть на Keras для малого табличного датасета load_wine (задание 2) и свёрточная сеть на Keras для задачи компьютерного зрения (задание 3). Во всех трёх задачах были успешно применены методы регуляризации Dropout и BatchNormalization, а также механизм EarlyStopping с восстановлением лучших весов. Установленные в заданиях пороги точности достигнуты и превышены во всех случаях: лучшая модель задания 1 показала 89.30%, лучшие модели задания 2 – 100.00% (при пороге 94%), а CNN-модель задания 3 – 96.97% (при пороге 85%). Проведённое исследование подтвердило ключевые теоретические положения: важность простых архитектур как отправной точки, эффективность Dropout как универсального регуляризатора и необходимость применения CNN в задачах обработки изображений.